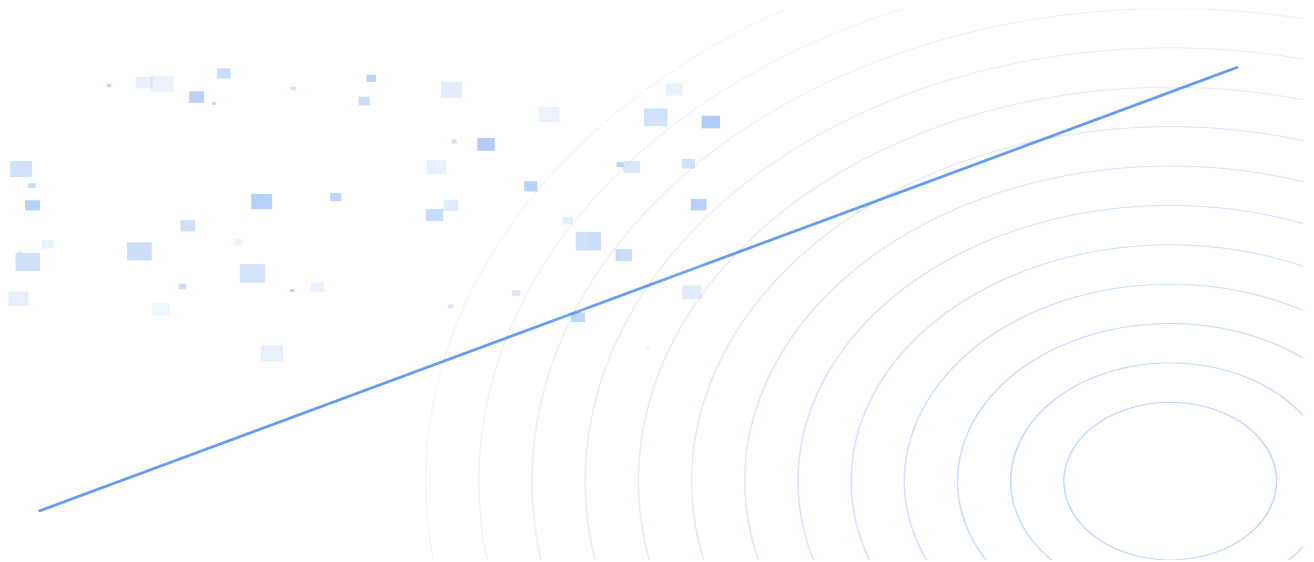




ISHU RAKESHBHAI SINGH

AI / Automation Engineer · InSAR & Geospatial · Full-Stack Builder

singhishu2060@gmail.com · +995 555 114 467 · Tbilisi, Georgia · ishu-portfolio.vercel.app

I build AI-assisted software and the plumbing around it — production web apps, Telegram-driven workflow automations, healthcare platforms, and scientific data pipelines. Comfortable across Python, TypeScript and JavaScript; comfortable on AWS; happiest when something needs to actually ship and stay up. Recently I picked up InSAR / Sentinel-1 SAR processing from scratch and delivered a strict EGMS-L3 equivalent road-subsidence pilot on a 3-year stack — the kind of project that mixes a lot of engineering with quite a bit of careful science.

AI / Automation

Multi-workflow n8n, OpenAI tool calling, Anthropic SDK, RAG via Pinecone.

InSAR / Geospatial

Sentinel-1 SLC pipelines in SNAP + MintPy + SNAPHU; ITRF14 PMM tie; ERA5 atmospheric correction.

Healthcare IT

Live 4-portal hospital system, GPT-4o Copilot with tool calling, Gynae/Obstetrics-shaped schema.

CONTENTS

00	About me
00	How I work
01	Haimcore — Next.js AI workspace with Amirani assistant
02	Relief Guru — Telegram-driven AI avatar video pipeline
03	Jalaram Hospital — AI hospital management system, four portals
04	SATALITE Delhi — Strict EGMS-L3 road-subsidence pilot
05	SATALITE Kite Beach — L2-equivalent single-track Sentinel-1 pilot
--	Tech I lean on
--	Training & certifications
--	Glossary
--	Working with me

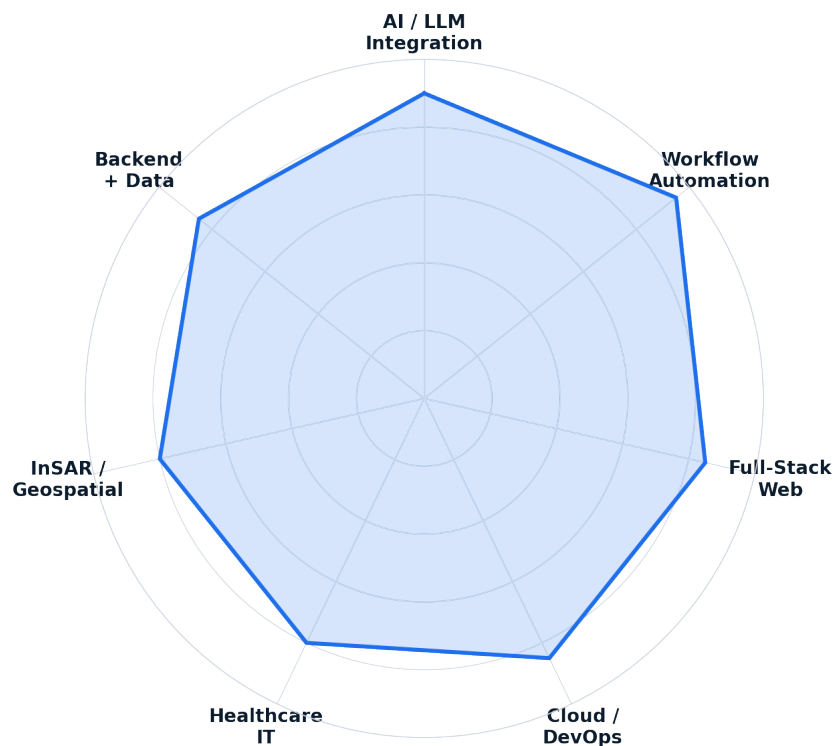
Click any entry to jump to it. Project pages start with a full-page intro — the meat is in the “What I built” section below it, and the “Bits that bit me” section is where the work actually shows.

About me

I build AI-assisted software and the plumbing around it — production web apps, Telegram-driven workflow automations, healthcare platforms, and scientific data pipelines. Comfortable across Python, TypeScript and JavaScript; comfortable on AWS; happiest when something needs to actually ship and stay up. Recently I picked up InSAR / Sentinel-1 SAR processing from scratch and delivered a strict EGMS-L3 equivalent road-subsidence pilot on a 3-year stack — the kind of project that mixes a lot of engineering with quite a bit of careful science.

Where my time goes

Self-rated proficiency across the seven categories I touch most often. The shape is the point — AI / automation and geospatial are both load-bearing right now.



How I work

- **Bias to shipping.** I'd rather get a v0 in front of the user this week and learn what's actually wrong than spend a month polishing the v3 of someone's spec.
- **Defensive at the seams, light in the middle.** Where my code meets a third-party API or an LLM, I hard-fail with the actual thing that broke. Inside the code I trust myself.
- **Audit logs over comments.** Useful comments are about *why*. But what I really care about is being able to reconstruct what happened — SystemLog on hospital, run trackers on Relief Guru, audit folders per release on Haimcore.
- **Hedging is honest.** If a parameter I've picked won't generalise outside the AOI / customer / case I had in front of me, I say so in the report. Better than the client finding out the hard way.
- **Make the un-fun parts repeatable.** Deploy scripts, smoke tests, rollback paths, idempotency guards. Everything that bit me once gets a guard rail.

01

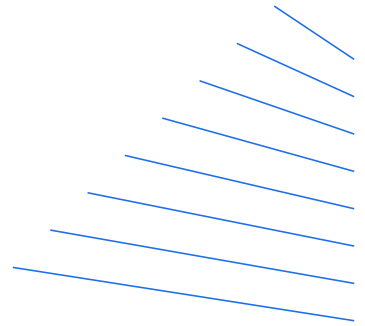
/ 05

AI PRODUCT

Haimcore

Next.js AI workspace with Amirani assistant

An operational AI-assisted workspace for the Haimcore team. Users dump content into drafts, folders, notes, and specialised editors; Amirani — the in-product assistant — summarises across it and answers questions in context.



PROJECT 01 / 05 · HAIMCORE

Lead AI software engineer · freelance · 2025 – 2026

3

MAJOR RELEASES

47

LANGUAGES SUPPORTED

v4

UX OVERHAUL

0

DOWNTIME ROLLBACKS

WHAT I BUILT

I built this as a Next.js application (pages router) with the API routes doubling as the back-end. The workspace is folders and drafts on the left, an editor in the middle, an Ask-About panel on the right. Specialised editors for text, URLs (with an Open-Graph fetch that I had to put a small SSRF guard around), images (vision-model description), and a “Note Mode” that attaches notes to a specific folder or file.

Amirani is wired straight to Anthropic’s SDK. Two small headaches I had to work around: the shell environment on the EC2 host had an empty `ANTHROPIC_API_KEY=` exported that was shadowing my `.env.local`, and a stale `ANTHROPIC_BASE_URL=` from an older proxy experiment. I wrote a tiny `resolveAnthropicKey()` in the legacy shim that reads `.env.local` directly as a fallback and forces the base URL to `api.anthropic.com` — saved me from ever debugging that one again.

There’s a small *insight* route that the image describer and Ask-About panel both post into. It appends to `data/memory/summary_default_<date>.md` so the next Amirani call sees today’s AI-generated content as context. Closes the loop without me having to design a full memory store.

The legacy-shim pattern is worth describing because it has saved me from a class of bugs. Anything that used to live at `pages/api/.../foo.js` moves to `lib/legacy/foo.js`, behind a thin v4 wrapper that adds CSRF + session handling. Each shim is exercised by the smoke test on every deploy, so an import-path break in a forgotten file shows up before the deploy lands, not after.

Auto-save is a 1.5second idle trigger plus a final sync save on editor unmount. There’s a `onDraftRebind` subscription so when a new draft swaps from a temporary optimistic ID to its real DB id, every component holding that reference updates without re-rendering or losing focus.

Multilingual: 47 languages including aliases (so a user can type the language in their own keyboard and the menu still matches). Came up in the v3 release as a hard requirement from a real client.

Deploy story: tagged pre-deploy snapshots, source-only bundles uploaded to S3, a Postgres rollback instance kept around for the v4 cut-over as a safety net, a manual `pre-rollback-backup-<date>` folder before any destructive change to source. I keep the rollback DB alive for at least 7 days after a cut-over — long enough that a real user’s “wait, where did that draft go?” lands while I still have the data to restore it from.

Habits I lean on. I never let two of my own folders share a prefix (so `haimcore-may-7` and `haimcore-may-7-backup` never exist together — one becomes `haimcore-prod-snapshot`). I commit the `AUDIT.md` before every release even when nothing changed, so the diff itself documents the release. And I stash a daily zip of the working tree outside the project dir because the Recycle Bin is not a backup — I learned that one the hard way when a 729 MB Downloads folder disappeared and I had to recover it before the night was over.

“I’ve had to fall back to the snapshots once and it was worth the discipline.”

THE BITS THAT BIT ME

After the v4 file overlay, the Anthropic SDK install lost `models.mjs` and a couple of beta resources — the streaming endpoint then returned an unhelpful 500. Reinstalling `@anthropic-ai/sdk@latest` fixed it, but I now reinstall the SDK whenever I overlay a release rather than trusting the file copy.

Two legacy shims (`system-health.js` + a missing `workspace-synthesize.js`) had stale `../../lib/X` imports after their move. The build went red, the cut-over paused. After fixing them I wrote a smoke check that exercises every shim before declaring a deploy green.

Recovered a 729 MB Downloads folder from the Recycle Bin during an incident-response session. Worth saying out loud: I now stash a daily zip of `haimcore/` outside the working tree, because the Bin is not a backup.

STACK

[Next.js \(pages\)](#)[Node.js](#)[TypeScript](#)[Anthropic SDK](#)[Prisma + PostgreSQL](#)[AWS EC2 + S3 + RDS](#)[PowerShell deploy](#)[PORTFOLIO](#)[GITHUB](#)[EMAIL](#)

02

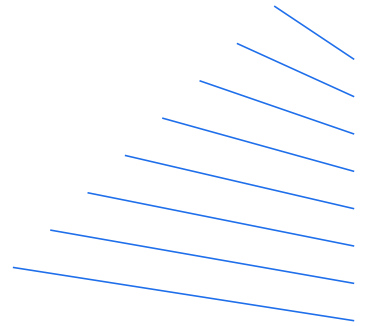
/ 05

AUTOMATION

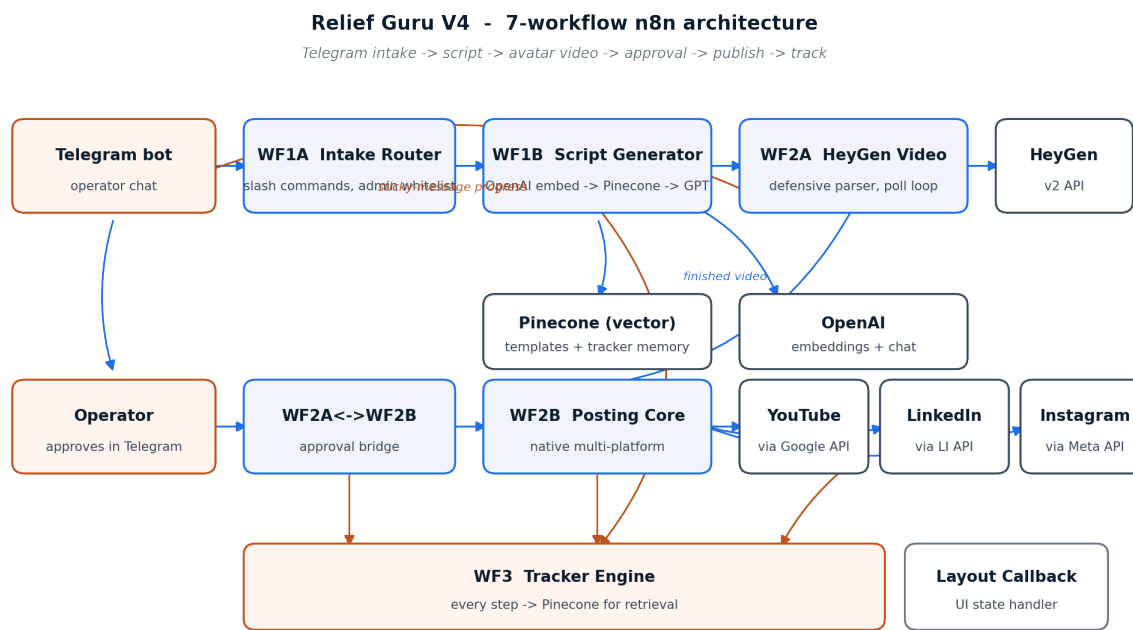
Relief Guru

Telegram-driven AI avatar video pipeline

A pharmacy-content business needed to publish AI avatar videos across YouTube, LinkedIn and Instagram, controlled from Telegram so non-developers can run it. I built that pipeline as seven coordinated n8n workflows.



AI automation engineer · freelance · 2025 – 2026



WHAT I BUILT

WF1A — Intake Router. Telegram webhook into n8n. Admin whitelist by chat ID. Slash-command parser: /start, /help, /avatars, /voices, /templates, /new, /refresh_avatars, /refresh_voices, /status, /last, /company. Routes the request into the right downstream workflow with a stable run_id.

WF2A — HeyGen Video Generation. The hairy one. Studio app posts a payload that, depending on which trigger fired, has the script under `body.script`, `body.video_inputs[0].voice.input_text`, `body.text`, or somewhere else. I wrote a *Parse Studio Payload* node that probes every reasonable path and, as a last-ditch, walks the whole payload looking for a string field with a name like `script / text / input_text` longer than 20 characters. If it doesn't find one it hard-fails locally with the keys it saw — way better than letting HeyGen respond with an opaque “Bad request” 90 seconds later. Same approach for `avatar_id` and `voice_id` — guard on length ≥ 6 before HeyGen ever sees it.

Idempotency guard. n8n's static workflow data stores `run_id` → `timestamp` for in-flight runs. A duplicate `run_id` within 30 minutes throws. Cleared lazily on every entry. Prevents Telegram retries (or the operator clicking a button twice) from spawning a second HeyGen render.

Sticky messages. Rather than spamming the chat with "recording... rendering... uploading... posted", the workflow edits a single Telegram message in place. Same `message_id` for the lifecycle of the task. Looks like a progress bar in the chat.

HeyGen poll loop. 30-second cadence, max 40 attempts (~20 minutes). The script-cleaning step strips zero-width characters, smart quotes, HTML tags, markdown, and stage-directions in brackets/parens/braces. Truncates at the last sentence end below the 1500-character HeyGen cap.

WF2A↔WF2B — Approval Bridge. Once HeyGen is done, the operator approves in Telegram. Approval token is signed; no spoofing.

WF2B — Posting Core (native). Google Developer Console for Drive + YouTube; LinkedIn Developer Console for the LinkedIn post; Instagram via Meta API. Each posting is its own retry-safe sub-flow.

WF3 — Tracker Engine. Every step in every run logs a row with `task_id`, `trace_id`, `log_type`, `workflow_id`, `step`, `status`, `state`, `embed_text`. The `embed_text` (first 500 chars of the script if available, otherwise the topic) goes back into Pinecone so the next run can retrieve "what did we say about this topic last time".

Habits I lean on. I label every code-node revision inside its own comment header (`rev1 / rev2 / dated`) so when the operator messages me at 11pm about a posting failure I can read the file and remember what I last changed without re-running anything. I also keep one "dead-letter" channel in Telegram that *only* the workflows can post to — anything that throws lands there with the run-id, so the operator doesn't have to know what n8n is to forward me a useful bug report. Acceptable for this operator; would almost certainly need a real dashboard for a bigger team.

"V1 was one giant n8n workflow. Splitting it into seven single-purpose webhooks made every failure traceable to a single workflow ID."

THE BITS THAT BIT ME

HeyGen used to fail silently because `input_text` was empty — the script was nested deeper in the payload than the parser knew about. After two of these I rewrote the parser to be paranoid and label each rev in the comments. Last revision is `v2026-05-03 rev2`.

Telegram's *Edit Sticky* HTTP node returns its own `{ok, result, ...}` object, which then becomes `$json` for the next code node. I lost ~30 minutes before realising the "script missing" error was actually the upstream context being shadowed. Now *Clean Script* explicitly reaches back to *Code: Idempotency Guard* and *Code: Parse Studio Payload* by name if `$input` doesn't have what it expects.

Operationally, the seven-workflow split is more code but less rework. The whole point is that the next time *posting to LinkedIn* breaks at 11pm, the operator sees "WF2B failed" in Telegram instead of a 60-node n8n trace.

STACK n8n (V4) Telegram Bot API HeyGen v2 API Creatomate OpenAI embeddings Pinecone (vector)
Google & LinkedIn Developer Consoles JavaScript + JSON



PORTFOLIO



GITHUB



EMAIL

03

/ 05

HEALTHCARE

Jalaram Hospital

AI hospital management system, four portals

An AI-assisted hospital management system for a women's health clinic. Four portals (Patient, Doctor, Staff, Admin), a tool-calling clinical AI Copilot, daily GPT-4o insights for doctors, and a data model shaped around Gynaecology and Obstetrics. Live in production.

PROJECT 03 / 05 · JALARAM HOSPITAL

Full-stack engineer · freelance · 2025 – 2026 · live at shreejalaramhospital.live

4	12	12	GPT-4o
USER PORTALS	CONTROLLERS	PRISMA MODELS	AI CORE

WHAT I BUILT

Four portals, one backend. *PatientPortal* — the patient's view: appointments, medical records, prescriptions, lab reports, menstrual-cycle tracker. *DoctorDashboard* — today's roster, full consultation form (vitals, GPAL counts, pregnancy module, pelvic exam notes, advice), prescription builder + per-doctor templates, AI insights, Copilot chat. *StaffDashboard / Ops Center* — the front desk: live counts of scheduled / completed / missed appointments, pending and overdue follow-ups, reports filed today, call attempts logged with notes. *AdminDashboard* — users, audit log, system-wide metrics.

JALARAM SUPER-AI Copilot. GPT-4o with tool calling against the Postgres schema. Two tools: `search_patients` (by name or by PAT-1-2026-format hospital ID) and `get_patient_context` (the full clinical bundle for a given patient). The system prompt locks the assistant's identity, names it explicitly, and says authorisation is handled at the login layer — the model is told not to refuse for "privacy" reasons inside the app, because the JWT has already done the role check. Each doctor's chats live in `CopilotSession` with auto-titled threads and message history in `CopilotMessage`.

Doctor AI Insights. Every morning the doctor opens the dashboard and the backend runs a roster anonymisation pass: counts of today's appointments by status, follow-up queue size, overdue follow-ups, irregular-cycle alerts (cycle length below 21 days or above 35). All PII stripped — only patient IDs make it into the prompt. GPT-4o is asked for exactly three actionable bullets in a strict, HIPAA-flavoured clinical tone, returned as HTML `<li>` elements so the front-end renders them directly. Token use is bounded by the deliberate summarisation upstream.

Gynae / Obstetrics-shaped data model. Custom Prisma schema: `PatientProfile` with hospital ID format `PAT-<seq>-<year>`, GPAL counts (gravida, para, abortions, living children), `isPregnant`, EDD and pregnancy notes; `MenstrualCycleLog` with start/end dates, flow intensity, symptoms, cycle length, period duration, predicted next-period / ovulation / fertile-window dates; `ClinicalConsultation` with vitals (BP, pulse, weight, temperature), LMP and flow + pain levels, chief complaint, obstetrics history, contraceptive method, trimester + risk category, general and pelvic examination, suggested reports and advice; `OperationRecord` with surgeon, assistant, anaesthesia type and complications. There's also a `SystemLog` table that every controller writes to via a `recordLog()` helper, capturing the actor, action, details and IP. That's the audit trail the admin sees.

Reports + lab uploads. Multer-S3 to AWS S3 in the ap-south-1 region, signed URLs back to the front-end. The `Report` table has an `aiSummary` field — the next iteration auto-summarises uploaded ultrasounds and lab reports back into the patient's timeline. Lazy-init OpenAI / S3 clients so the env is read at call time and a misconfigured deploy fails loudly instead of silently using a stale key.

Production hardening. Express 5 + TypeScript + Prisma 6 + Postgres. JWT in HTTP-only cookies, bcrypt password hashing, helmet for security headers, a CORS whitelist that includes `shreejalaramhospital.live` and `localhost` for development. 20 MB JSON body limit so report uploads with embedded base64 thumbnails still parse. Request logger middleware on every route. Two date helpers because dashboards compute "today" in IST: `getTodayBounds()` and `getTomorrowBounds()` round-trip through the +5h 30 offset rather than trusting Date defaults.

Deploy. Docker + docker-compose + Nginx (TLS) + PM2 on EC2, with Prisma migrations baked into the start script. `deploy_to_live.ps1` on the local side, `setup_ec2.sh` + `start_pm2.sh` on the box. Frontend served via a small `serve_frontend.cjs` bridge for Vite's built output.

Habits I lean on. Every date that crosses a clinical boundary goes through `getTodayBounds()` / `getTomorrowBounds()` — I do not trust raw Date arithmetic on a server whose clock is UTC but whose doctors live in IST. I added an explicit `recordLog()` call to every state-changing controller, even the ones that “couldn't possibly go wrong”, because the admin's most common question is “who edited this record at 3pm?” and the only honest answer comes from a log row. Two-thirds of my time on this project was schema + audit, not UI.

“The system prompt locks the assistant's identity and says authorisation is handled at the login layer — the JWT has already done the role check.”

THE BITS THAT BIT ME

Role logic in the controllers is the easy thing to get wrong. When a STAFF or ADMIN creates a medical record or prescription on behalf of a doctor, the supervising `doctorId` has to come from the request body, not the caller's JWT. DOCTOR-role creates pull from JWT. Wrapped that branch in the controller and added a 400 if STAFF/ADMIN forgets to send the doctor's ID — saved me a class of “prescription signed by reception” bugs.

Cycle-irregularity alerts are noisy by default — 21/35-day thresholds are textbook but pull in too many patients on a real roster. The dashboard currently flags them but I've been tempted to add a per-doctor configurable band; haven't shipped that yet because the doctors actually want to see the long tail.

The OpenAI calls were originally constructed at module load. If `process.env.OPENAI_API_KEY` wasn't set yet (e.g. PM2 starting before `.env` was fully sourced), the client was permanently broken until restart. Moved both OpenAI and S3 clients to lazy getters — `getOpenAI()` / `getS3Client()` — that re-read `env` at call time. Fixed by inspection, never had to debug it under load.

STACK Express 5 + TypeScript Prisma 6 + PostgreSQL React + Vite + TypeScript OpenAI GPT-4o (tool calling)
AWS S3 (multer-s3, ap-south-1) JWT + bcrypt + helmet Docker + Nginx + PM2 AWS EC2
pdfkit / pdf-parse



PORTFOLIO



GITHUB



EMAIL

04

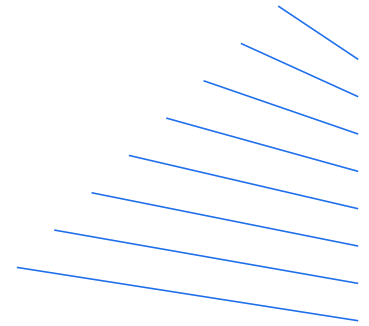
/ 05

INSAR

SATALITE Delhi

Strict EGMS-L3 road-subsidence pilot

A strict EGMS-L3 equivalent road-subsidence map — vertical and east-west deformation rates per pixel, ITRF14 absolute frame, ERA5 tropospheric correction, two-track decomposition. End to end on a local Linux workstation with open-source tools.



PROJECT 04 / 05 · SATALITE DELHI

InSAR / remote-sensing engineer · freelance · April – May 2026 · Najafgarh Road, SW Delhi

2,555

PERSISTENT SCATTERERS

476

SBAS PAIRS

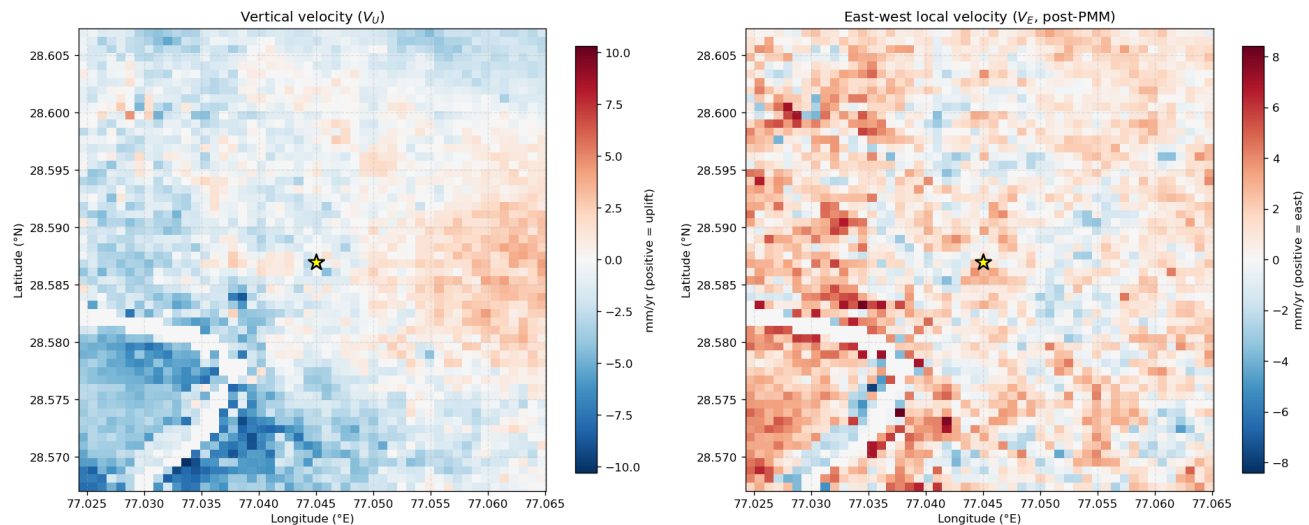
~2 mm

RESIDUAL RMS

~0.95

DECOMP DETERMINANT

Strict EGMS-L3 decomposition — Najafgarh Road, Sector 8 Dwarka, SW Delhi



Source: [velocity_dual_panel.png](#)

WHAT I BUILT

The site choice itself was the first piece of work. I went into this expecting to pilot at Visakhapatnam — coastal subsidence is the obvious story for an Indian road. A 30-second ASF catalog check killed that: no ascending Sentinel-1 coverage at all. Without ASC + DSC you can't do strict EGMS-L3 decomposition. Mumbai and Bangalore had the same gap. That left Delhi and Kolkata as the two dual-track-covered Indian metros I knew of. I picked Delhi — well-documented groundwater subsidence in southwest Delhi in the literature, inland (no coastal geoid effects to fight during atmospheric correction), and Najafgarh Road in Sector 8 Dwarka is real road infrastructure the client can point to.

Data: 165 Sentinel-1 SLCs across 3 years. ASC relative orbit 27, IW2, 88 acquisitions. DSC relative orbit 136, IW1, 77 acquisitions. ~12-day cadence (S1A only — S1B failed end-2021 and S1C didn't come online for the Delhi pass until mid-2025, so the late-window frames are sparse and I treated the whole stack as 1A-cadence for the SBAS network design). The geometric point that mattered later: ASC sits west of the AOI looking east, DSC sits east looking west — antiparallel horizontal LOS vectors with similar incidence angles, ~33°. That's what makes the 2×2 $V_U + V_E$ decomposition well-conditioned.

Workstation and software. 12-core / 32 GB / 2 TB SSD on Ubuntu 22.04. SNAP for the per-pair InSAR work — bumped the JVM heap from the default 8 GB to 20 GB in `~/snap/etc/snap.properties` because the default crashes on coregistration of large bursts. Python 3.10 conda env with MintPy + SNAPHU + `asf_search` + `sentineleof` + `rasterio` + `geopandas` + `pyproj` + `h5py` from conda-forge; PyAPS3 from GitHub (it's not on conda-forge). I never touch `h5py` manually after the one time I mixed pip and conda installs and segfaulted MintPy at `load_data`.

SBAS network. Each acquisition pairs with the three nearest forward neighbours, max temporal baseline 60 days. Past 60 days Delhi's monsoon decorrelation is too severe in my experience to be worth processing. 258 ASC + 218 DSC = 476 pairs.

SNAP per pair. TOPS coregistration with ESD, interferogram formation, topo phase removal against Copernicus 30m DEM, deburst, 35×6 multilook (~80m pixels), Goldstein filter, SNAPHU unwrap, terrain correction to UTM 43N. Built once in SNAP Graph Builder, saved as XML, run via gpt for every pair. I split the graph into two halves around the SNAPHU export/import — SNAP's built-in SNAPHU wrapper stalled silently on three different pairs early in the run with no useful log entry, and running SNAPHU directly on the command line at least lets me see when it's thrashing on tile boundaries.

ERA5 atmospheric correction. Tropospheric water vapour adds ~1–2cm peak-to-peak LOS noise over Delhi across a year. Without correction it bleeds into the velocity as a fake annual cycle, especially for any pixel near vegetation. I batched CDS requests overnight — 01:00 UTC for descending (~06:00 local pre-dawn), 13:00 UTC for ascending (~18:00 local). About 300 GRIB files. PyAPS3 expects a very specific filename pattern; a couple of files from an earlier test had the wrong hour suffix and MintPy silently re-downloaded them. Renaming saved the download time.

MintPy SBAS. Separate workspaces `mintpy_asc/` and `mintpy_dsc/`. Mixing both into one workspace works in principle but debugging is painful once the geometry files start overlapping. `modify_network` dropped 5 ASC pairs + 1 DSC pair against `minCoherence = 0.5`, preserving the MST. `reference_point` landed on a high-coherence rooftop on the AOI's eastern edge in both stacks (same physical structure — reassuring). `correct_troposphere` with the ERA5 step is the slow one, ~25 minutes per stack. Residual RMS settled at ~2mm in both stacks; EGMS noise floor is ~2–3mm so I called this good enough.

ITRF14 tie via the Indian-plate Euler pole. MintPy gives relative velocities (reference pixel = 0). EGMS L3 wants absolute ITRF14. With no co-located GNSS at the AOI I used Altamimi et al. (2017)'s plate-motion model to compute the expected ITRF14 velocity at my reference rooftop and subtracted it from every pixel. Pole at $\phi = 50.295^\circ\text{N}$, $\lambda = -8.155^\circ\text{E}$, $\Omega = 0.554^\circ/\text{Myr}$; at my AOI that's $V_E \approx +40$ and $V_N \approx +39$ mm/yr, $V_U = 0$ by rigid-plate assumption.

Validation against MIDAS-IGS14. Three GNSS stations — LCK4 (Lucknow, 427 km), HYDE (Hyderabad, 1251 km), IISC (Bangalore, 1732 km). Horizontal residuals coherent at the -4 to -6mm/yr east and -4mm/yr north level across a 1,700km span. Almost certainly a small bias in the Altamimi 2017 Indian-plate pole (likely tightened up in ITRF2020 — I didn't redo the calculation with the 2020 pole because by the time it occurred to me I was already through decomposition; one for a future run). The LCK4 vertical residual is well-documented Lucknow groundwater subsidence, not a frame issue.

Decomposition. Sentinel-1's polar orbit gives the V_N coefficient under 5% of total, so dropping V_N is the standard move. Solved the remaining 2×2 (V_E , V_U) system per-pixel by Cramer's rule; the per-pixel determinant rides at ~0.95 across the AOI (anything above 0.3 is usable, 0.95 is comfortable). Wrote the determinant out as its own raster for the QA dashboard tab.

PS mask. Average spatial coherence ≥ 0.70 in **both** stacks — the EGMS rule. Mean ASC 0.83 (range 0.53–0.97), mean DSC 0.88 (0.55–0.98). Pixels passing in both: **2,555 PS**. I tested 0.75 and dropped to ~2,100 — tighter felt like wasted observations, the AOI was already cleanly populated at 0.70.

Deliverables. Vertical and east-west velocity rasters on the 80m grid in ITRF14, GeoJSON + CSV of the 2,555 PS with the full attribute set (lat, lon, V_U , V_E _local, V_{LOS_asc} , V_{LOS_dsc} , temporal coherence per stack, height, decomposition determinant, residual RMS), ~438k-row time-series CSV, static map figures, three markdown reports plus matching PowerPoint decks, a Streamlit dashboard with eight tabs (Map / Time Series / Profile / Decomposition QA / Statistics / SBAS Network / Data Explorer / Method).

Habits I lean on. I keep ERA5 outside the MintPy workspace — `/data/sataLite/era5/` rather than under `mintpy_asc/` — because `smallbaselineApp.py` will happily delete and re-download its working ERA5 cache if the config changes, and a re-download from CDS during European business hours can sit in the queue for an hour. I keep ASC and DSC in fully separate workspaces for the same reason — mixing them works in principle but debugging anything is painful once the geometry files start overlapping. And I lock the SNAP version for the whole pilot duration; mid-run a 9 → 10 minor bump would have meant editing my MintPy

load patterns mid-stream, which I've done once and never want to again.

"Acceptable for this AOI; would not generalise."

THE BITS THAT BIT ME

Multilook calibration. First batch was 30×5 looks. A few pairs in I noticed the result over the road corridor was smeared — small-target PS getting averaged into their neighbours. Reran a subset at 35×6 (~80×84m), visibly cleaner, locked it in. EGMS L3 is at 100m formally; 80m gave me a slightly finer grid without exploding compute.

SNAP JVM OOM. The pattern was always the same: a large burst pair, four gpt processes running in parallel, OS killing the JVM. After the second time I capped parallel at three concurrent pairs and the crashes stopped. Throughput dropped from ~5 pairs/hour to ~4. I lived with it.

ESD threshold on a few pairs. Default ESD coherence threshold is 0.3. A handful of pairs failed there; I dropped to 0.2 for those specific pairs and they coregistered fine. Not ideal — a lower threshold means trusting a noisier azimuth-shift estimate — but the AOI itself was high-coherence even though the broader burst had a low-coherence stretch. Acceptable for this AOI; would not generalise.

ERA5 zero-byte GRIBs. Three GRIB files came back zero bytes from the CDS endpoint and tropo_pyaps3 segfaulted on one of them three days later, when MintPy tried to read it. Error was uninformative — "Segmentation fault" with no traceback. I now eccodes-check every GRIB after download and run a `cfgrib.open_dataset()` sanity probe before anything reaches MintPy.

Mid-stack shape mismatch. Pair 20240623_20240705, a monsoon-window pair, had a slightly different raster size after deburst — probably because SNAP's auto burst-selection caught a different sub-burst due to a slight orbit shift. Regenerated that one pair separately with explicit `firstBurstIndex` / `lastBurstIndex` matching the rest of the stack and the rerun was clean. I didn't dig further once it worked.

2024 SAFE-layout shift. ASF's cloud-native rollout in 2024 changed the inner directory layout of the newer SLCs slightly. SNAP's default Read node initially refused to recognise a handful of the newer scenes. A small file-pattern tweak in the SNAP graph fixed it — lost ~an hour figuring out it wasn't a corrupt download.

STACK

Sentinel-1 SAR

SNAP / ESA STEP

MintPy SBAS

SNAPHU

PyAPS3 / ERA5

ITRF14 / Altamimi 2017 PMM

Streamlit + Plotly

GeoPandas + GDAL

AWS EC2 (dashboard)

Python 3.10



PORTFOLIO



GITHUB



EMAIL

05

/ 05

INSAR

SATALITE Kite Beach

L2-equivalent single-track Sentinel-1 pilot

First InSAR pilot for the SATALITE brief. Sentinel-1 road-subsidence monitoring at Kite Beach Road, Visakhapatnam. Descending-only by necessity — the site sits in a well-known Sentinel-1 ascending-coverage gap over coastal east India. This is the pilot that validated the pipeline before the strict-L3 Delhi run.

PROJECT 05 / 05 · SATALITE KITE BEACH

InSAR / remote-sensing engineer · freelance · April – May 2026 · Visakhapatnam, Andhra Pradesh

152

PERSISTENT SCATTERERS

244

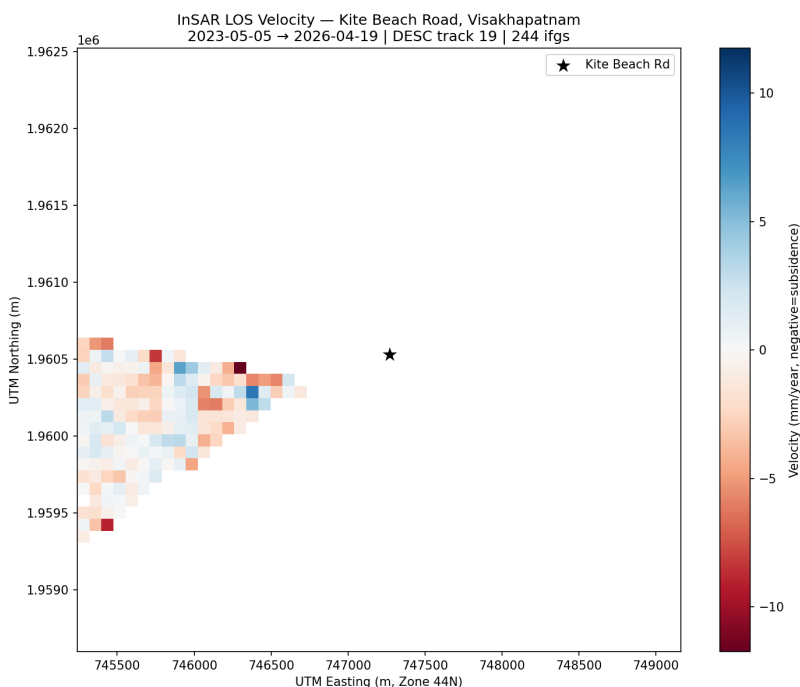
SBAS PAIRS

~5 mm

RMSE

86

SLC BURSTS



Source: velocity_map.png

WHAT I BUILT

Single-track by site geometry, not by choice. An exhaustive ASF API probe across every product type and both S1A and S1B for 2014–2026 returned **zero** ascending granules over a 3-year window at this AOI. The honest framing in the client doc: this is L2-equivalent (LOS-only) under EGMS terms, not strict L3 (V_U + V_E). Stated up front — saves the client the time of asking.

Stack and method. 86 Sentinel-1 SLC bursts, descending relative orbit 19, IW1, VV polarisation, May 2023 – April 2026. SBAS network with the same EGMS-style rule as Delhi (3 nearest forward neighbours, max 60-day temporal baseline). 250 pairs submitted to ASF HyP3 originally; 244 retained after coherence filtering at the load step.

MintPy SBAS time-series inversion. Same workflow scaffolding I'd later upgrade for Delhi — `smallbaselineApp.py` with config tuned to the AOI, reference pixel on a stable rooftop, coherence-based network modification with `keepMinSpanTree`. 152 persistent scatterers at temporal coherence ≥ 0.85 . LOS velocity range -11.7 to $+8.5$ mm/yr; mean -0.71 mm/yr; time-series RMSE ~ 5 mm (within EGMS noise floor).

Most-subsiding PS at 17.7174 N, 83.3222 E — west of the road, built-up area. Flagged for client follow-up.

Pilot deliverables — deliberately scoped. Velocity map, cumulative subsidence map, time-series chart at the most-subsiding PS, GeoJSON of the 152 PS, time-series CSV, summary JSON, plus a Streamlit dashboard. Two markdown reports (pilot report + methodology). Numbered Python scripts in `scripts/` covering the full pipeline end to end — this script chain is what got reused and upgraded for the Delhi strict-L3 run.

Habits I lean on. Before quoting any new InSAR site now, the very first thing I do is run `20_check_track_coverage.py` against ASF's API for both S1A and S1B across the full Sentinel-1 archive. It takes 30 seconds and prevents the most expensive mistake on a pilot like this: quoting strict L3 for a site that physically can't support it. I'd rather have an awkward 30-second conversation up front than deliver a methodology PDF that the client's scientist sees through three weeks later.

Limitations called out in the client doc (deliberate, addressable in the full project): single descending track only (no ascending → no full 3D decomposition), no ERA5 atmospheric correction applied yet (would reduce noise ~50%), local reference frame (not tied to ITRF). All four of those got upgraded for Delhi, by design.

"Stated up front — saves the client the time of asking."

THE BITS THAT BIT ME

The ASF gap is the real story. I burned time on the catalog check up front and it was the right call — running the full pipeline blind would have produced single-track output with no way to flag for the client that the strict L3 deliverable was geometrically impossible at this site. The catalog check became a one-time `20_check_track_coverage.py` script I now run before quoting any new InSAR site.

Burst SLC display. A side artifact — the `sentinel1_burst_2026-04-19.tiff` demo file is raw SLC complex data (Cint16, complex64, no map projection). It will not display as a normal radar image in QGIS without amplitude extraction + georeferencing. A reminder that not every `.tiff` on an InSAR project is a raster you can open and pan.

STACK [Sentinel-1 SAR](#) [ASF HyP3](#) [MintPy SBAS](#) [SNAPHU](#) [Streamlit + Plotly](#) [GeoPandas](#)
[AWS EC2 \(dashboard\)](#) [Python 3.10](#) [QGIS](#)



PORTFOLIO



GITHUB



EMAIL

TECH I LEAN ON

Languages	Python · TypeScript · JavaScript · Java
AI & LLMs	OpenAI APIs (GPT-4o, embeddings, tool calling) · Anthropic SDK · LangChain · Pinecone (vector) · RAG · Prompt engineering
InSAR / Geospatial	Sentinel-1 SAR · SNAP / ESA STEP · MintPy SBAS · SNAPHU · PyAPS3 / ERA5 · ASF HyP3 · ITRF14 / Indian-plate PMM · GeoPandas · GDAL · QGIS · Streamlit
Cloud & ops	AWS (EC2, S3, RDS, IAM, Lambda) · Docker · docker-compose · Nginx · PM2 · Serverless
Backend	Node.js · Express 5 · FastAPI · Next.js (pages) · REST · JWT · bcrypt · helmet
Frontend	React · Vite · TypeScript · Next.js · Streamlit · Plotly
Databases	PostgreSQL · Prisma 6 ORM · MongoDB · Redis · Firebase · Pinecone (vector)
Automation	n8n (multi-workflow architectures) · Telegram Bot API · Make.com · Zapier · Webhooks · Event-driven pipelines
Integrations	OpenAI · Anthropic · HeyGen · Creatomate · Pinecone · Google Workspace APIs · LinkedIn Developer APIs · ASF Vertex · ESA Copernicus · Telegram Bot API
Other	Git · PowerShell · pdfkit / pdf-parse · Multer-S3 · Helmet / CSRF / cookie auth · Agile

TRAINING & CERTIFICATIONS

AI & Machine Learning Specialisation

Generative AI, neural networks, applied ML (Udemy, Coursera, Campus X)

LangChain & Generative AI Development

Production LLM applications and patterns (Educative + industry training)

AWS Cloud Fundamentals & Serverless Architecture

EC2 / S3 / RDS / IAM / Lambda, infrastructure design

Full-Stack Web Development

Node.js + Python + React across multiple frameworks

Workflow Automation & Integration Platforms

Hands-on n8n / Make.com / Zapier across real client work

Healthcare IT Systems Architecture

HMS / EHR data models, HIPAA-aware workflows

Sentinel-1 InSAR Processing

SNAP / MintPy / SNAPHU pipelines, EGMS-L3 methodology

GLOSSARY

Short reference for the InSAR / healthcare jargon that appears in the project pages.

AOI	Area of Interest. The geographic region a pilot focuses on.
ASC / DSC	Ascending / descending satellite passes. Sentinel-1 collects each AOI in both, viewed from opposite sides — a prerequisite for decomposing motion into vertical and horizontal components.
SBAS	Small Baseline Subset. An InSAR processing technique that pairs acquisitions with short temporal and spatial baselines to invert for a time-series of ground displacement.
PSI / PS	Persistent Scatterer Interferometry. The reliable measurement points after coherence filtering — typically buildings, rocks, and infrastructure. EGMS publishes its products as PS.
EGMS	European Ground Motion Service. Copernicus's ground-deformation product for Europe; L3 refers to the strict, decomposed ($V_U + V_E$) tier.
LOS	Line-of-sight — the satellite's viewing direction. Single-LOS observations resolve motion along that axis only; two LOS directions resolve $V_U + V_E$.
ITRF14	International Terrestrial Reference Frame, 2014 realisation. Defines absolute ground positions and velocities globally.
PMM	Plate Motion Model. Adds the expected rigid-plate motion at the AOI (here from Altamimi 2017's Indian plate pole) so the relative MintPy velocities can be tied to ITRF14.
SNAP	Sentinel Application Platform — ESA's open-source SAR processing toolkit. Used for the per-pair interferogram chain.
MintPy	Python toolbox for InSAR time-series analysis. Reads SNAP outputs and inverts the SBAS network.
SNAPHU	Statistical-cost, Network-flow Algorithm for Phase Unwrapping. Resolves 2π ambiguities in the interferogram.
PyAPS3	Python Atmospheric Phase Screen library. Calls ECMWF reanalysis (ERA5) to estimate and remove tropospheric delay from InSAR.
GPAL	Gravida / Para / Abortions / Living children — standard obstetric history counts used on the consultation form.
EDD	Expected Date of Delivery. Stored per pregnancy.
RBAC	Role-based Access Control. Patient / Doctor / Staff / Admin in the hospital schema.
HIPAA	US health-data privacy standard. The hospital AI prompts strip PII down to patient IDs in a HIPAA-flavoured tone — the clinic is in India, not legally HIPAA-bound, but the model is kept on the safer side regardless.

CSRF	Cross-Site Request Forgery. The v4 wrappers on the Haimcore legacy shims add a CSRF token check.
SSRF	Server-Side Request Forgery. The Open-Graph fetch on the URL editor was a candidate target; guarded with an allowlist.
RAG	Retrieval-Augmented Generation. The pattern Relief Guru uses for templates — embed the brief, retrieve the closest template from Pinecone, then generate against it.

WORKING WITH ME

I'm open to remote and freelance engagements globally. Most comfortable on AI-assisted product work, automation pipelines, full-stack TypeScript / Python, and geospatial processing — ideally where some part of the problem hasn't been solved cleanly before and the engineering judgement matters as much as the code.

Email singhishu2060@gmail.com

Phone +995 555 114 467

Based in Tbilisi, Georgia

Portfolio ishu-portfolio.vercel.app

GitHub github.com/ishunteam-png



EMAIL



PORTFOLIO

Languages English (Fluent), Hindi (Native), Gujarati (Native), Georgian (Conversational)

